

537 notebook — operator manual

This note describes `537_Sports_Simulation.ipynb`, `sports/sim_config.py`, and how promotion, pools, and binning relate to each other. For implementation details, see `CELL 1` in the notebook (helper definitions).

Quick start

1. Edit `sports/sim_config.py` (population size, pools, seeds, defaults).
2. In Jupyter: run **Cell 0** (optional), **Cell 1** (helpers), **Cell 10** (widgets). **Cell 11** (optional): PPM histogram from `datasets/mbb/player_season_panel_530.csv`.
3. Cell 10 calls `reload_sim_config()` at the top; re-run it after saving `sim_config.py` to refresh globals and widget sync when using batch lock.

Batch diagnostic cells (2–9) were dropped from the live notebook on 2026-05-08. A freeze of the earlier version lives at `obsolete_files/sports_gameplan_old/537_Sports_Simulation_pre_archive_8MAY.ipynb`.

Core objects (per simulated person)

Symbol	Meaning
N (<code>N_INDIVIDUALS</code>)	People per run.
K (<code>N_WINNERS</code>)	Promotion slots per run ("draft picks", "tenure lines", etc.).
N_POOLS	Number of teams / pools. People are partitioned across pools (approximately equal roster sizes when using the equal-size template).
A or A_i	Latent ability for person <i>i</i> , drawn each run from the chosen ability law (A/B/C in config or widget). Clipped or scaled to [0, 1] depending on law.
pool_id	Integer team label (only when pools are on).
sorting_signal	Signal used only to assign pools: <code>A_i + ε_i</code> with <code>ε_i ~ Normal(0, SORTING_NOISE_SD^2)</code> if noise > 0; else equals A_i . True A_i is unchanged.
global_rank_score	Percentile rank of A within the run (larger = better).
local_rank_score	Percentile rank of A within (run, pool_id).
pool_quality	Pool mean of A (reported on each row; mean of A in that pool for that run).
promoted	Boolean: selected as a winner that run.

Ability distribution (`ABILITY_DISTRIBUTION_CHOICE` / widget)

Code	Widget label	Draw
A	Uniform(0,1)	<code>Uniform(0, 1)</code> .
B	clipped Normal(0.5, 0.18)	<code>Normal(0.5, 0.18)</code> clipped to [0, 1].
C	scaled Beta(2,5)	<code>Beta(2, 5)</code> then scaled by max so support matches notebook helper.

Promotion score modes (`score_mode` / widget "Promotion score")

Weights passed into **winner draw** are built from:

Mode	Weights (conceptually)
ability	<code>A_i</code> (not the main widget focus).
global_rank	Global rank score (no-pool reference world).
local_rank	Local rank score within pool (fixed internal mix of rank signal when this mode is selected from dropdown).
local_rank_plus_ability	<code>w * local_rank_score + (1 - w) * A_i</code> with slider <code>ADDITIVE w</code> (<code>ADDITIVE_LOCAL_RANK_WEIGHT</code> from config or widget).

Winner draw (`WINNER_SELECTION_CHOICE` / widget "Winner draw")

Code	Label	Mechanism
A	Weighted K without replacement	Normalized nonnegative weights → <code>numpy.random.Generator.choice(..., size=K, replace=False, p=p)</code> .
B	Bernoulli	Each person promoted independently with probability capped by <code>min(normalized_weight * K, 1)</code> (see notebook <code>choose_winners</code>).
C	Top-K	Deterministic: highest K weights win.

Anyone with **true** $A_i < \text{MIN_ABILITY_FOR_PROMOTION}$ gets promotion weight **0** before the draw, so they cannot be selected. If too few people pass the cutoff in a run, **fewer than K** may be promoted for modes **A** and **C**.

Pool assignment (`pool` / `LOCAL_POOL_ASSIGNMENT` patterns)

Uses `sorting_signal` (noisy or exact **A**):

Code	Pattern
A	Random equal-sized pools (permutation of a round-robin template).
B	Assortative: sort by signal, fill pools in order so similar signals share a pool.
C	Disassortative: sort by signal, assign to pools in modulo fashion to mix highs and lows.

Caption under plots switches to “noisy assortative / disassortative” when **SORTING_NOISE_SD** > 0.

Cell 10: widgets (reference)

Order is **top** → **bottom** as in the notebook UI (roughly: global options, then plot/bins, then simulation scales, then **Run / refresh**).

UI label (ipywidgets description)	Control	Maps to / internal	Persist key (sports/cell10_playground_state.json)
EDA blurb	HTML	—	—
Batch lock: N, runs, bins, pool bins, min A promote ← sim_config	Checkbox	INTERACTIVE_USE_MAIN_SCALES default	use_batch
Binning	Dropdown	individual_qcut / pool_equal_count / pool_equal_width	binning_mode
Plot (x-axis)	Dropdown	pool_local / pool_global / pool_A / nopool_global	view
A _i distribution	Dropdown	A / B / C → ABILITY_DISTRIBUTION_CHOICE	ability
Winner draw	Dropdown	A / B / C → WINNER_SELECTION_CHOICE	winner
Min A to promote	FloatSlider [0, 1]	MIN_ABILITY_FOR_PROMOTION	min_A_promote
Sorting noise sd	FloatSlider	SORTING_NOISE_SD ; max from INTERACTIVE_SORTING_NOISE_MAX	noise
ADDITIVE w (local-rank share)	FloatSlider [0, 1]	ADDITIVE_LOCAL_RANK_WEIGHT ; disabled when promotion score is Local rank only	additive
Runs (playground)	IntSlider	INTERACTIVE_N_RUNS ; batch lock → N_RUNS	runs
N individuals	IntSlider	INTERACTIVE_N_INDIVIDUALS ; batch lock → N_INDIVIDUALS	n_indiv
Person bins (#)	IntSlider	INTERACTIVE_N_BINS ; batch lock → N_BINS . Disabled unless Binning = Individuals	bins
Person x edges	Dropdown	equal_count / equal_width → summarize_by_bins(..., person_binning=...) . Disabled unless Binning = Individuals	person_x_binning
Pool–talent bins (K)	IntSlider	INTERACTIVE_N_POOL_AGG_BINS ; batch lock →	pool_bins

UI label (ipywidgets description)	Control	Maps to / internal	Persist key (sports/cell10_playground_state.json)
		N_POOL_AGG_BINS . Disabled unless Binning is a Pools: mode	
Pool assignment	Dropdown	A / B / C (random / assortative / disassortative)	pool
Promotion score	Dropdown	local_rank / local_rank_plus_ability	score
Run / refresh	Button	Triggers redraw	—

Persistence: On each successful plot refresh, Cell 10 writes the knob dict to sports/cell10_playground_state.json (path relative to the kernel working directory; repo-root cwd is assumed—see notebook intro). Delete that file to restore pure sim_config defaults on the next Cell 10 run.

Batch lock: When checked, Cell 10 syncs N individuals, Runs, Person bins, Pool–talent bins, and Min A to promote from sim_config.py and disables those sliders until you unlock.

Cell 10: binning modes (how the x-axis is built)

A) Individuals: bin x (person-level rows)

- Binning** dropdown (Cell 10): **Individuals** = slice the plotted x into bins over all person-rows (local_rank_score , global_rank_score , or A).
- Person x edges** (Cell 10): **equal count** uses rank + pd.qcut ; **equal width** uses pd.cut on

$$\min(x), \max(x)$$

in summarize_by_bins — same **Person bins** count, different edge rule.

- Person bins** slider (or N_BINS when batch lock): number of bins on x.
- Plot: line** (plot_bin_summary): $x \approx$ bin mean of x, $y =$ mean promoted in bin.

B) Pools: equal pool count per bin (method="equal_count")

- Pool–talent bins K** (or N_POOL_AGG_BINS with batch lock).
- Within each run:** compute each pool’s mean A; **sort pools** by that mean; split pool IDs into K contiguous blocks with numpy.array_split (as close to equal number of pools per bin as possible).
- Constraint:** need N_POOLS ≥ K .
- Across runs:** average promotion rate and **mean leave-one-out peer A** in each bin (summarize_by_pool_mean_A_bins ; column poolq_loo per row — same role as empirical poolq_loo).
- Plot: line** (plot_pool_A_bin_summary): $x =$ mean poolq_loo in the pool–talent bin, $y =$ mean promoted . There is **no separate UI toggle** — choose **Binning** → a **Pools:** option in Cell 10 (and a **pooled** plot view).

C) Pools: equal width on mean A (`method="equal_width"`)

- Same K, but within each run build K equal-width intervals from min to max of pool mean A (`pd.cut`).
- Some intervals may contain zero pools in a given run; those bins vanish or carry no mass for that run—fewer than K points can appear after averaging.
- Use when you want bins comparable to **equal-width** talent bands rather than equal team counts.
- **Plot:** same as (B): x = mean LOO peer A in bin, not Q1...QK bar positions.

Config vs widget: who wins?

Knob	Primary location	Notes
N, K, N_POOLS, seeds, N_BINS, N_POOL_AGG_BINS, noise default, w default, min A for promotion	<code>sim_config.py</code>	Reload via Cell 0 / Cell 10 start or <code>reload_sim_config()</code> .
Batch lock	Cell 10 checkbox	When on: N, runs, both bin counts, min A to promote track <code>N_INDIVIDUALS</code> , <code>N_RUNS</code> , <code>N_BINS</code> , <code>N_POOL_AGG_BINS</code> , <code>MIN_ABILITY_FOR_PROMOTION</code> .
Ability, winner draw, pool mode, promotion score, w, noise, min A cutoff, plot view, binning mode	Cell 10 widgets	Override config for interactive EDA without editing file (except N_POOLS , K still from config unless you change file).
Figure size	<code>FIG_PLAYGROUND_EDA_INCHES</code> in <code>sim_config.py</code>	Inches (width, height) for the single EDA figure.

File map

Path	Role
sports/sim_config.py	<code>RUN_CELL1</code> , <code>RUN_CELL_PLAYGROUND</code> , <code>RUN_CELL11</code> , N, K, N_POOLS, N_BINS, N_POOL_AGG_BINS, <code>MIN_ABILITY_FOR_PROMOTION</code> , seeds, figure size, default A/B/C choices, sorting noise default, additive <code>w</code> default.
sports/537_Sports_Simulation.ipynb	Cell 1: helpers. Cell 10: playground widgets. Cell 11: optional PPM histogram from <code>player_season_panel_530.csv</code> . Optional state: <code>sports/cell10_playground_state.json</code> .
obsolete_files/sports_gameplan_old/537_Sports_Simulation_pre_archive_8May.ipynb	Snapshot before batch cells 2–9 were removed from the live notebook (2026-05-08).
sports/documents/537_Manual.md	This document.

Troubleshooting

Symptom	Likely cause
<code>NameError</code> for helpers	Run Cell 1 (<code>RUN_CELL1 = True</code>).
Pool binning + no pools	Choose a pooled plot view or switch binning to individuals.
"N_POOLS must be ≥ K"	Equal-count pool binning only; lower K or raise N_POOLS in <code>sim_config.py</code> .
Fewer than K pool points (equal width)	Some width bins empty in some runs—normal for equal-width on sparse pool means.

Version note

- Cell 10 no longer ships a **Cell 8a preset** (explicit sliders and `sim_config` only).
- On **2026-05-08** the live notebook dropped **CELL 2–9**; the frozen copy lives at `obsolete_files/sports_gameplan_old/537_Sports_Simulation_pre_archive_8May.ipynb`, and `sim_config.py` no longer exposes `RUN_CELL2` ... `RUN_CELL9`.